

CI/CD

Павел Галанин

Весна 2026



CI/CD: Зачем?

А какую проблему мы решаем?

90-е и 00-е: Эпоха ручного труда

● Месяцы работы в своих ветках →
«**Merge Day**»

- конфликты кода
- сломанная логика
- недели на починку

● Релиз = бессонная ночь, ручные скрипты, заклинания сисадмина

● «А у меня всё работает!»

Решение: *DevOps* и *Extreme Programming*

● **CI (Интеграция)**: Сливаем код каждый день.
Ошибка? Узнаем через 5 минут.

● **CD (Доставка)**: Релиз — это предсказуемая, скучная кнопка, а не подвиг.

● Единое, стандартизированное окружение сборки для всей команды.

А какую проблему мы решаем?

Для мобильной разработки ещё критичнее - слишком высока цена ошибки:

- Креш на старте → 1 звезда, пользователь удаляет приложение
- Откатить на предыдущую версию нельзя
- Доставка новой версии до пользователя занимает часы и даже дни

Что это?

CI/CD - одна из практик DevOps, которая заключается в комбинации непрерывной интеграции и непрерывной доставки

CI (Continuous Integration) — Непрерывная интеграция

Автоматическая сборка и проверка кода тестами при каждом пуше в репозиторий - гарантирующая, что новые изменения не сломали существующий проект.

CD (Continuous Delivery/Deployment) — Непрерывная доставка

Автоматизированный процесс упаковки успешно проверенного кода в релизный артефакт (APK/AAB) и его бесшовная передача тестировщикам или публикация в магазине приложений.

→ Разработчик только пишет код и делает git push. Остальное происходит автоматически

CI/CD

Плюсы

- проблемы интеграции выявляются и исправляются быстро, что оказывается дешевле
- немедленный прогон модульных тестов для свежих изменений
- постоянное наличие текущей стабильной версии вместе с продуктами сборки — для тестирования, демонстрации, и т. п.
- немедленный эффект от неполного или неработающего кода приучает разработчиков к работе в итеративном режиме с более коротким циклом.
- Уменьшение стоимости, времени и риска внесения изменений
- Уменьшение человеческого фактора

CI/CD

Плюсы

- проблемы интеграции выявляются и исправляются быстро, что оказывается дешевле
- немедленный прогон модульных тестов для свежих изменений
- постоянное наличие текущей стабильной версии вместе с продуктами сборки — для тестирования, демонстрации, и т. п.
- немедленный эффект от неполного или неработающего кода приучает разработчиков к работе в итеративном режиме с более коротким циклом.
- Уменьшение стоимости, времени и риска внесения изменений
- Уменьшение человеческого фактора

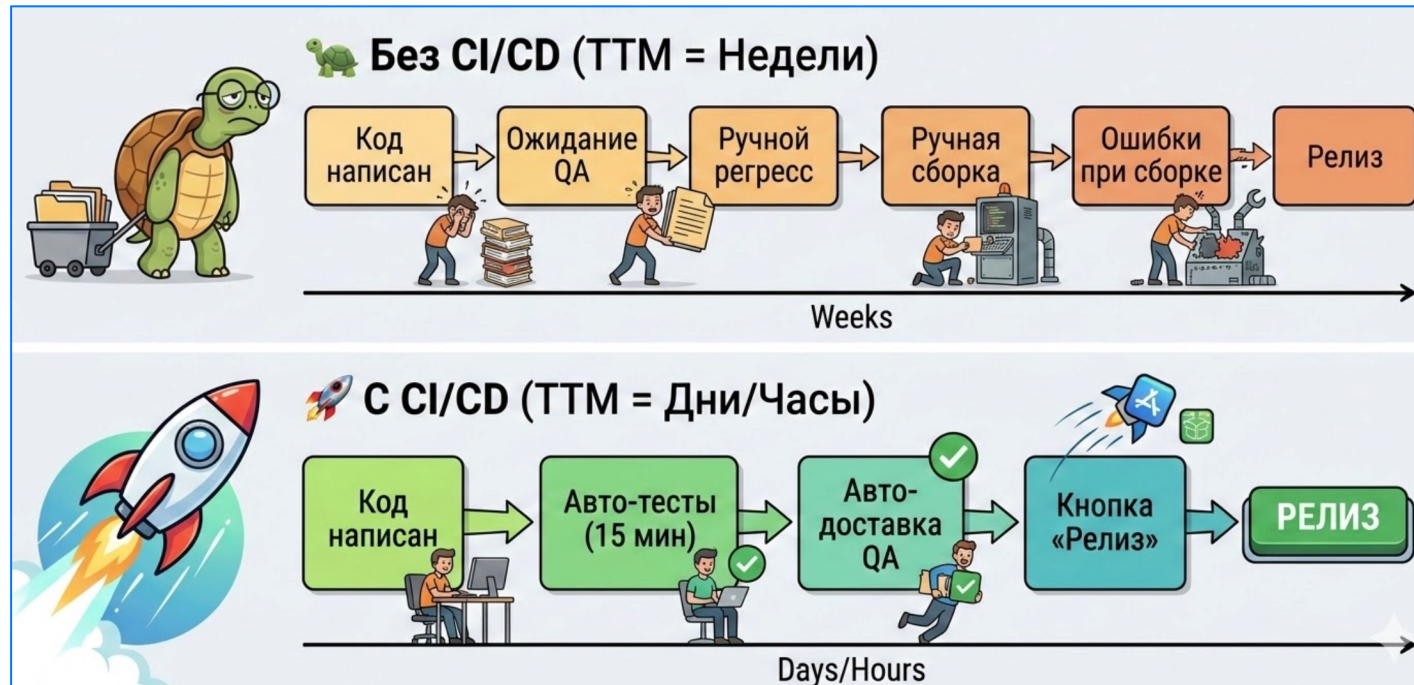
Минусы

- значительные затраты на поддержку работы непрерывной интеграции
- необходимость в дополнительных вычислительных ресурсах под нужды непрерывной интеграции
- порог входа (YAML-файлы, Docker-контейнеры, bash-скрипты и т.д.)

Time to Market

CI/CD положительно влияет на одну из важнейших метрик для бизнеса - **Time to Market (TTM)**: время от старта разработки фичи до момента, когда ею пользуются клиенты.

Быстрее проверяем гипотезы → быстрее получаем обратную связь → опережаем конкурентов



А мне это нужно?

Когда можно отказаться?

- Код пишется «в стол»: учебный/пет-проект/хакатон и т.д.
- Разрабатываете в одиночку, не обновляете/обновляете очень редко

Когда CI (CD) обязателен?

...

А мне это нужно?

Когда можно отказаться?

- Код пишется «в стол»: учебный/пет-проект/хакатон и т.д.
- Разрабатываете в одиночку, не обновляете/обновляете очень редко

Когда CI (CD) обязателен?

Во всех остальных случаях

CI/CD: Как?

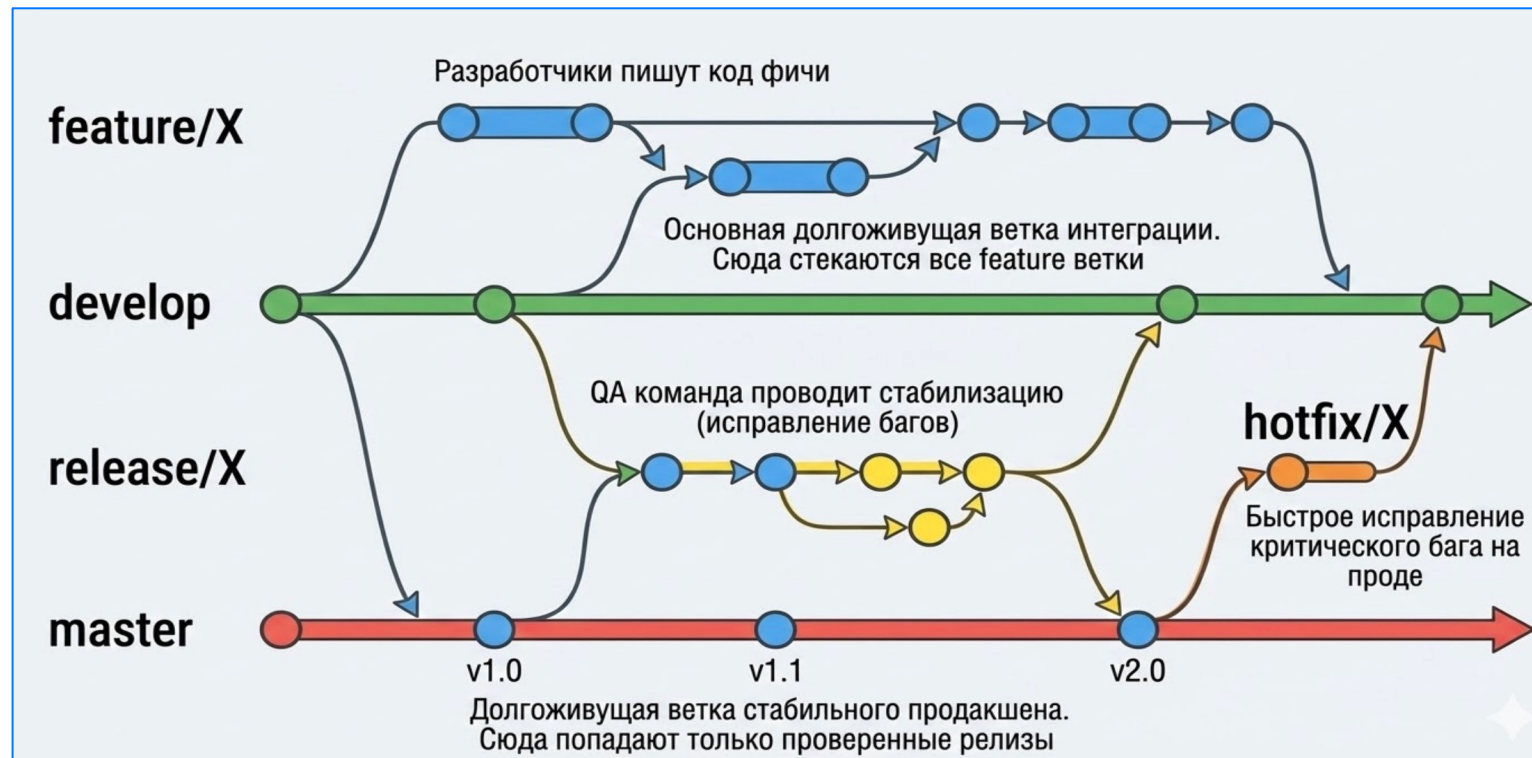
Стратегия ветвления в git

Чтобы CI/CD работал хорошо, нужно выбрать подход ведения git

- Git flow
- Trunk based development

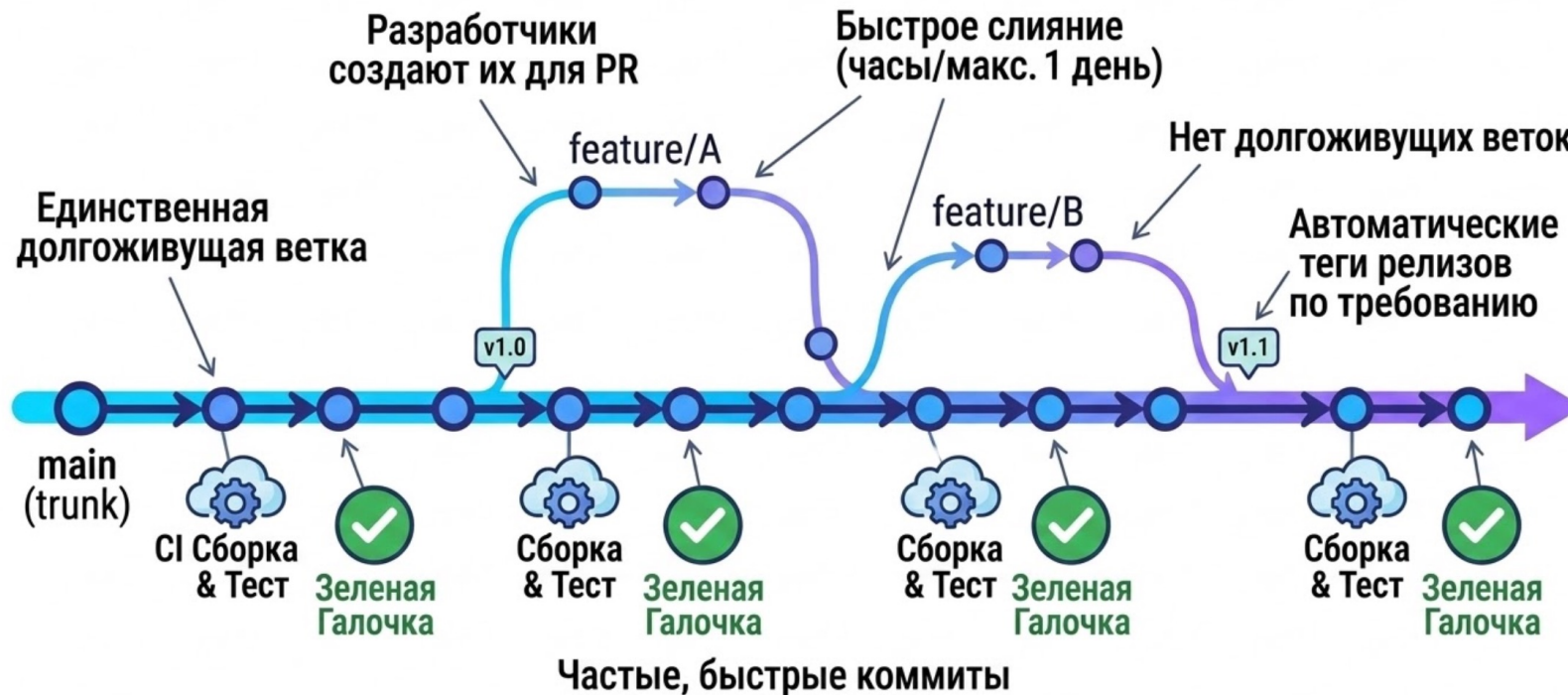
Git flow

- По отдельной ветке на каждую фичу
- В master - стабильные продакшн-версии
- Минусы: сложнее настраивать CI/CD, дольше интеграция



Trunk based development

- Короткоживущие ветки: не больше 1-2х дней
- Транк всегда готов к деплою, даже если в нем есть недописанные фичи
- Чёткое разделение продакшн-кода и разрабатываемого: Feature Flags, Branch By Abstraction
- Continuous Code Review
- Минусы: доп. разработка на изоляцию кода (флаги); незаконченный код в репозитории



Из чего состоит CI/CD

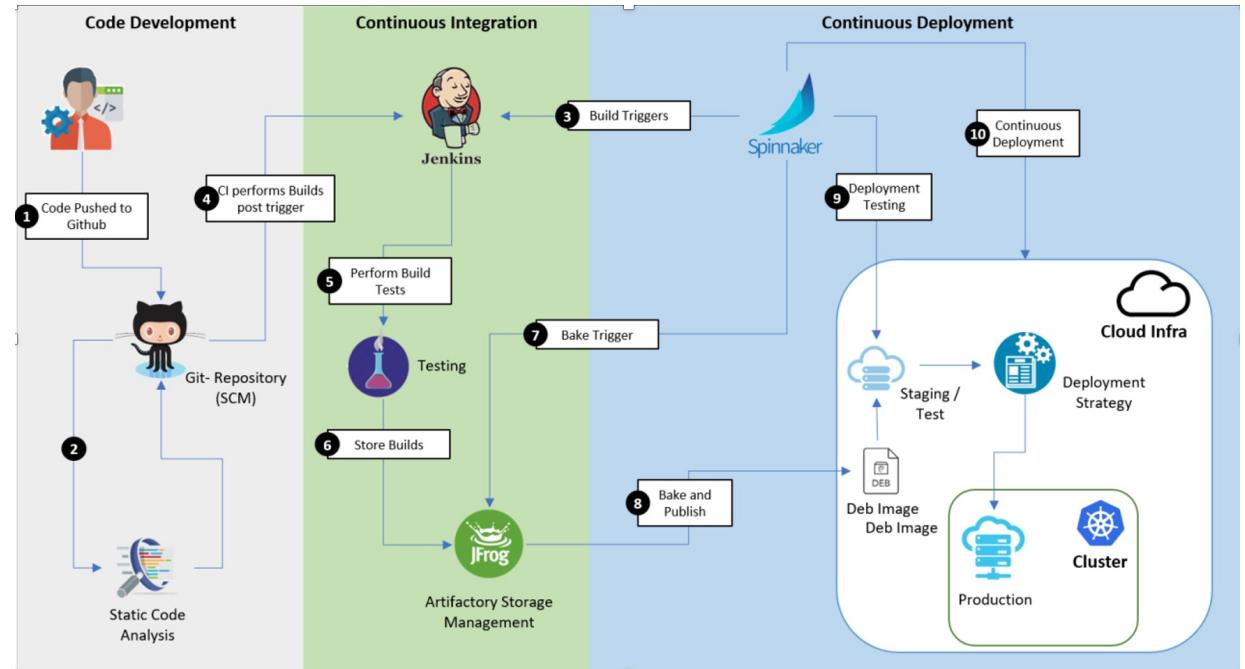
Pipeline (Пайплайн): Весь конвейер целиком.

Trigger (Триггер / Хук): Событие, которое запускает магию (Push в main, создание Pull Request, расписание).

Stage (Этап): Логический шаг конвейера (test, build, deploy). Этапы выполняются строго последовательно.

Job (Джоба / Задача): Конкретная работа внутри этапа. Джобы в одном этапе могут выполняться параллельно.

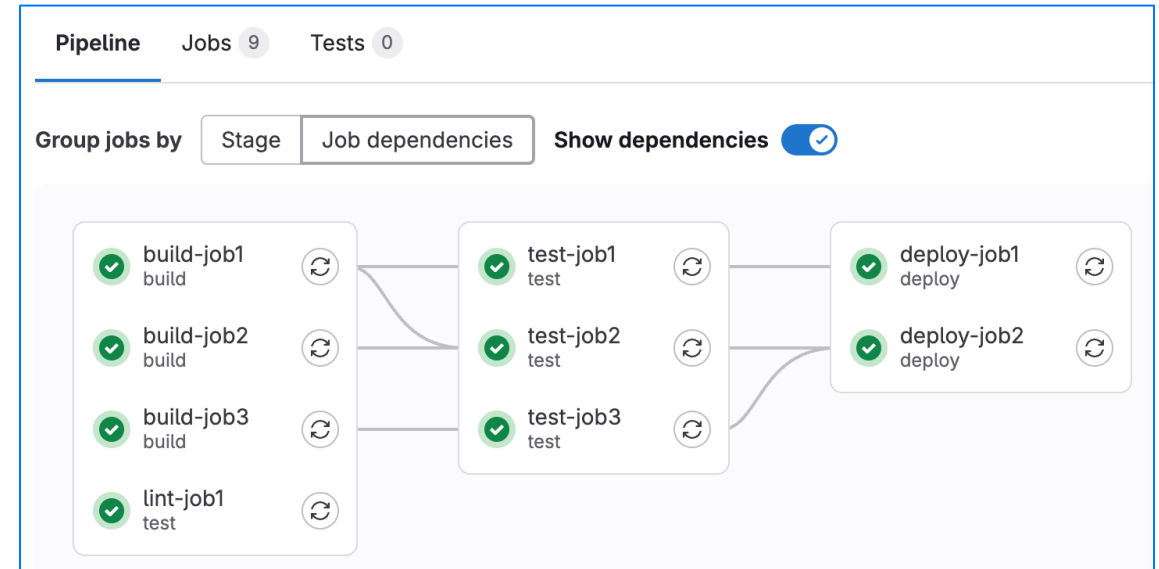
Runner (Раннер): Сервер/виртуальная машина, выполняющая джобы.



Stage

Типовые этапы:

- **Code Quality:** Lint, Detekt
- **Test:** Unit Tests, UI Tests
- **Build:** Assemble Debug, Bundle Release
- **Deploy:** Firebase App Distribution, Google Play
- **Notify:** Telegram/Slack bot



- Пайплайн строится по принципу «Fail Fast»: сначала легкие проверки, потом тяжелые сборки.
- Необязательно выполнять все этапы при каждом пуше.

Этап 1: проверки и тестирование

Статический анализ:

- Android Lint: Ищет баги Android API, проблемы с ресурсами. (`./gradlew lint`)
- Detekt (Ktlint): Проверяет стиль Kotlin-кода (отступы, нейминг). Не дает превратить код в спагетти.

Тестирование:

- Unit-тесты (JUnit/MockK): Проверяют бизнес-логику. Выполняются за секунды. (`./gradlew test`)
- UI-тесты (Espresso, Kaspreso): Имитируют клики пользователя. Требуют эмулятора. Боль: Очень медленные и нестабильные в CI.
- Решение для UI: Использовать облака вроде Firebase Test Lab.

Этап 2: сборка

`./gradlew build`

- `./gradlew assembleDebug`
- `./gradlew assembleRelease`

Этап 2: сборка

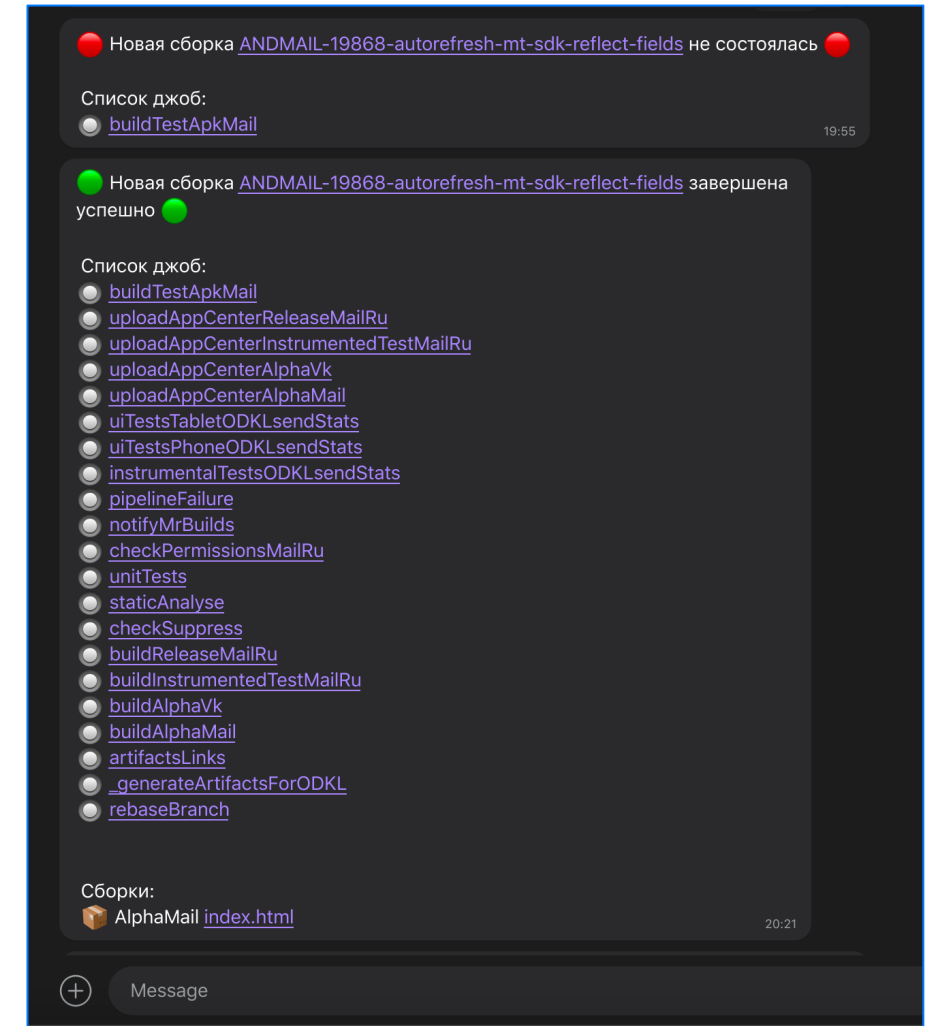
```
signingConfigs {  
    release {  
        storeFile file(System.getenv("KEYSTORE_FILE") ?: "debug.keystore")  
        storePassword System.getenv("KEYSTORE_PASSWORD")  
        keyAlias System.getenv("KEY_ALIAS")  
        keyPassword System.getenv("KEY_PASSWORD")  
    }  
}
```

Никогда не храните ключи в репозитории

Этап 3: деплой и уведомления

Публикация:

- Google Play Console
- RuStore
- AppGallery (Huawei)
- Firebase App Distribution
- Google Play Console (Internal / Closed треки)
- RuStore (Бета-тестирование)

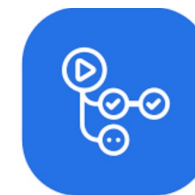


Инструменты и сервисы

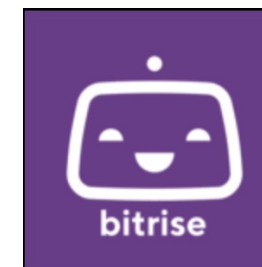
Инструмент	Порог входа	Для чего подходит	Плюсы	Минусы
GitLab CI	Средний	Enterprise, закрытые проекты	Мощный, встроен в репозиторий, Docker first	Требует настройки своих раннеров для тяжелых сборок
GitHub Actions	Низкий	Open-source, пет-проекты, стартапы	Огромная экосистема готовых шагов (Actions)	Дорогие минуты для приватных репозиториев macOS/Android
Bitrise	Самый низкий	Чисто мобильная разработка	Собирается мышкой, заточен под iOS/Android	Платный и довольно дорогой
Jenkins	Очень высокий	Огромные корпорации	Умеет абсолютно всё, бесплатно	Настраивается через боль, требует выделенного сисадмина



CI/CD



GitHub Actions



Jenkins

YAML-конфигурация CI/CD

- Gitlab CI: .gitlab-ci.yml в корне
- GitHub Actions: по пути .github/workflows/ с любым названием

Формат **YAML** (YAML Ain't Markup Language)

```
1  image: mingc/android-build-box:latest
2
3  ▾ stages:
4      - check    # Проверка кода (Lint)
5      - test     # Юнит-тесты
6      - build    # Сборка APK
7      - notify   # Отправка в Telegram
8
9  ▾ before_script:
10     - chmod +x ./gradlew
11
12  ▾ lint_job:
13     stage: check
14     ▾ script:
15         - ./gradlew lintDebug
16
17  ▾ unit_test_job:
18     stage: test
19     ▾ script:
20         - ./gradlew testDebugUnitTest
21
22  ▾ build_apk_job:
23     stage: build
24     ▾ script:
25         - ./gradlew assembleDebug
26
27  ▾ artifacts:
28     ▾ paths:
29         - app/build/outputs/apk/debug/app-debug.apk
30     expire_in: 1 day
```

Как сделать хорошо

- Стабильность CI
 - Если сломан CI, часть команды не работает
- Скорость сборки
 - Влияет на доставку артефактов
- Герметичность сборки
 - Избавляемся от внешних зависимостей
- Регулярные сборки
 - Запускаем регулярно и отслеживаем
- Документация сборки
 - Оставляем всем знания

Спасибо
за внимание!